



# **Operating Systems & Networks CTEC1704C**

## **Lecture - 7 : OS Structures, Designs and Files**

Dr. Sameer Jha, SFHEA  
Faculty of Computing  
De Montfort University Cambodia



# Outline

- Operating System
- Dual-Mode Operation
- Operating System Services
- System Calls
- OS Designs
- Files
- File and Directory Operations
- File System Layer



## What is an Operating System? – The Basics

A program that acts as an intermediary between a user of a computer and the computer hardware

Operating system goals:

- Execute** user programs and make solving user problems easier

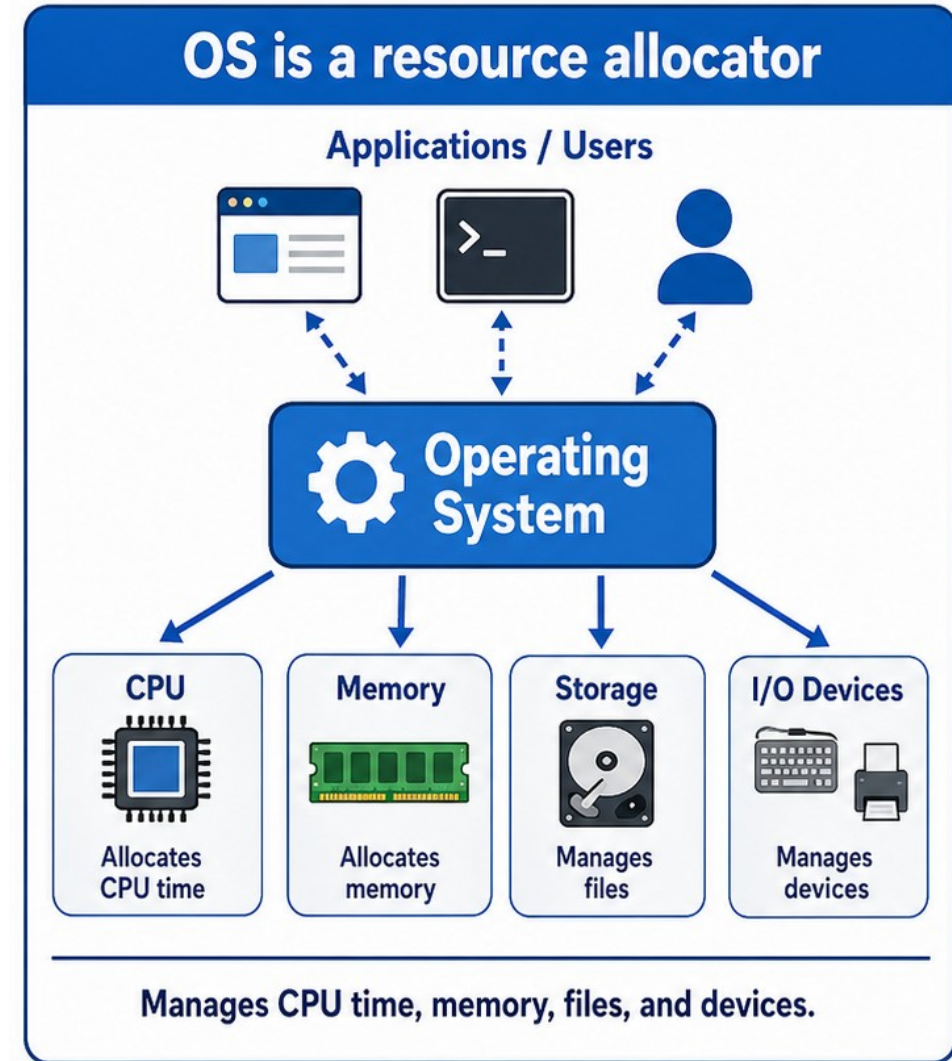
- Make the computer system **convenient** to use

- Use the computer hardware in an **efficient** manner

# Operating System Roles

## OS is a **resource allocator**

- Manages all resources
- Decides between conflicting requests for efficient and fair resource use

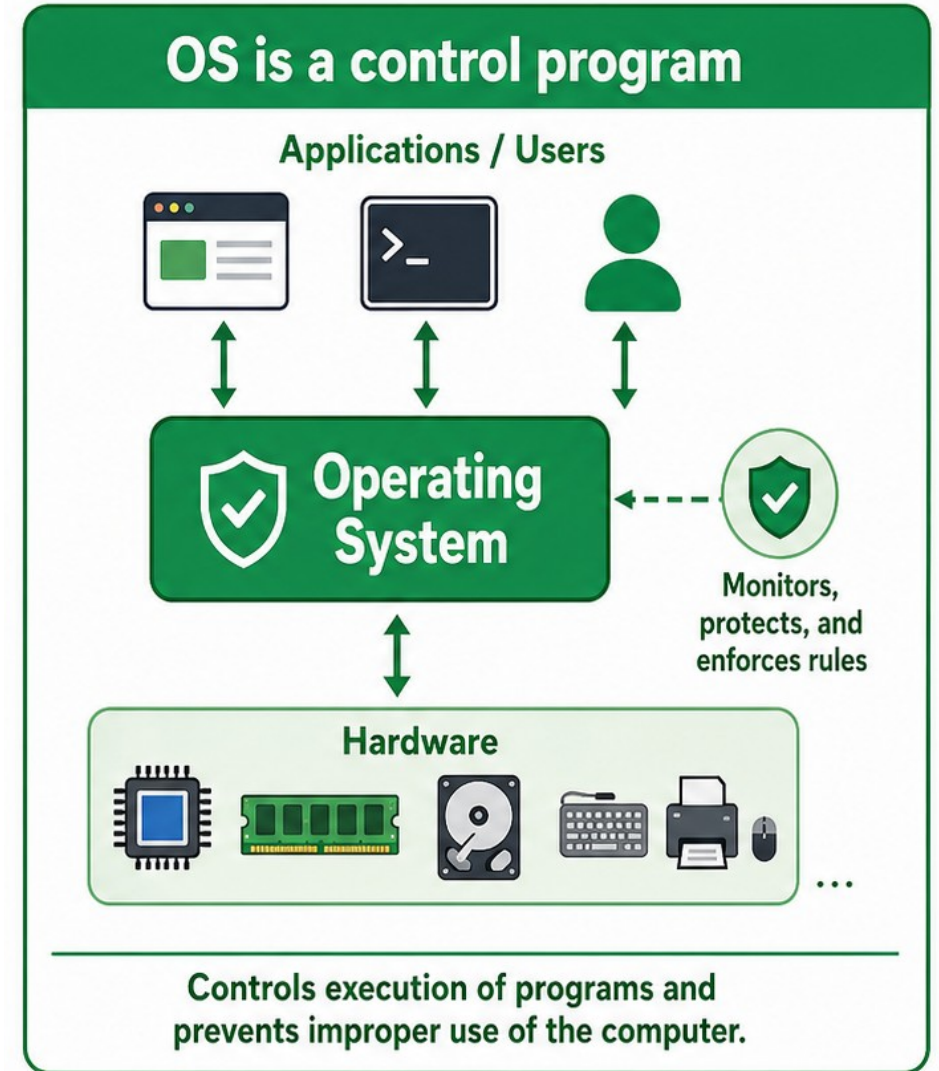




## Operating System Roles (Cont.)

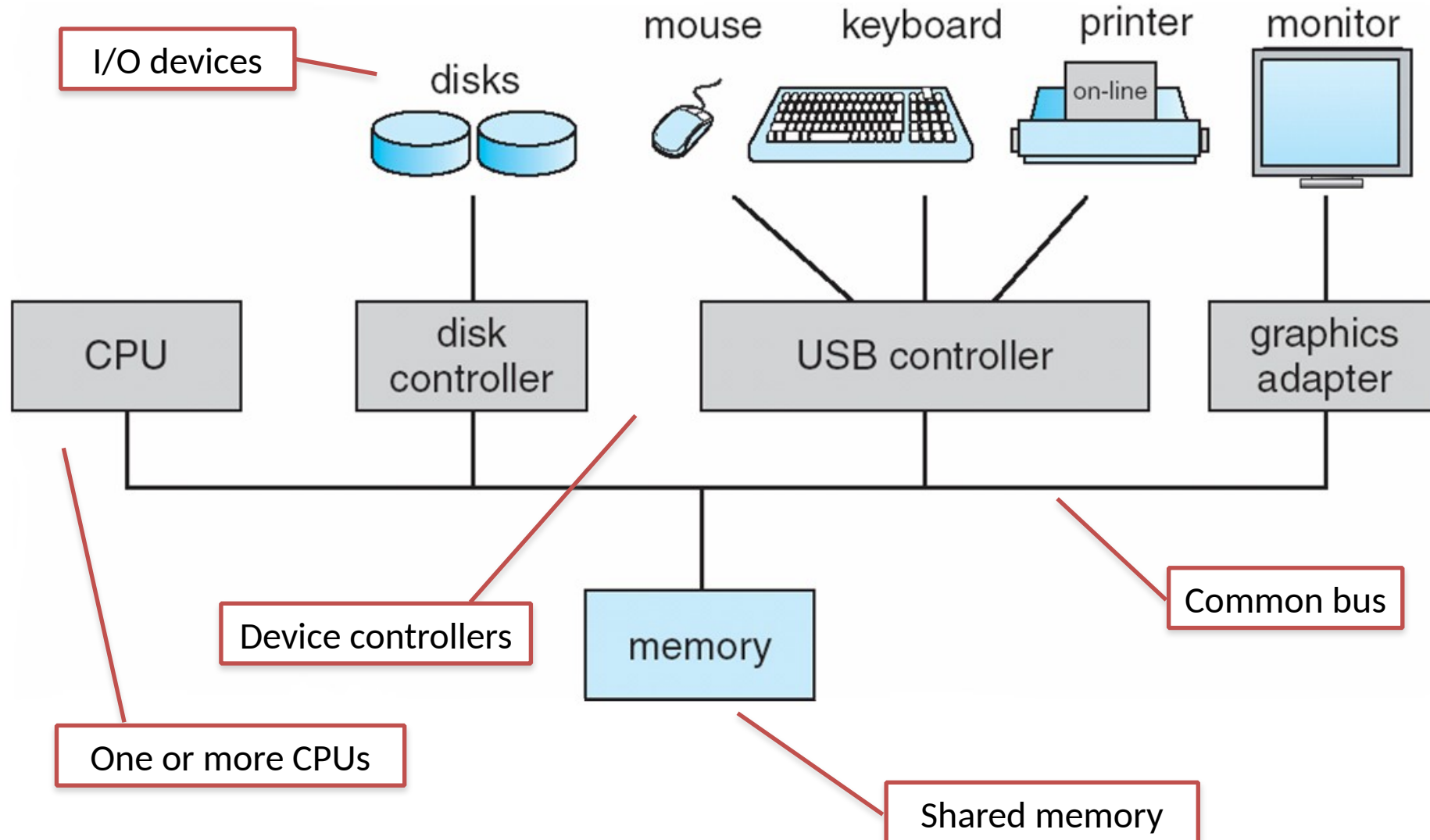
### OS is a **control program**

- Controls execution of programs to prevent errors and improper use of the computer





# Computer System Structure

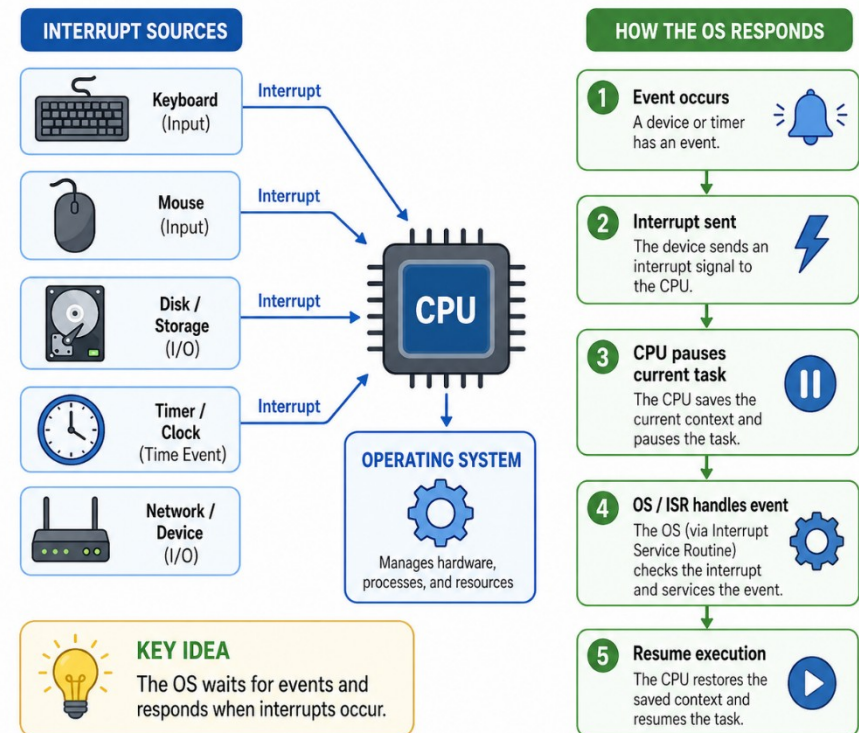




# Interrupts (1)

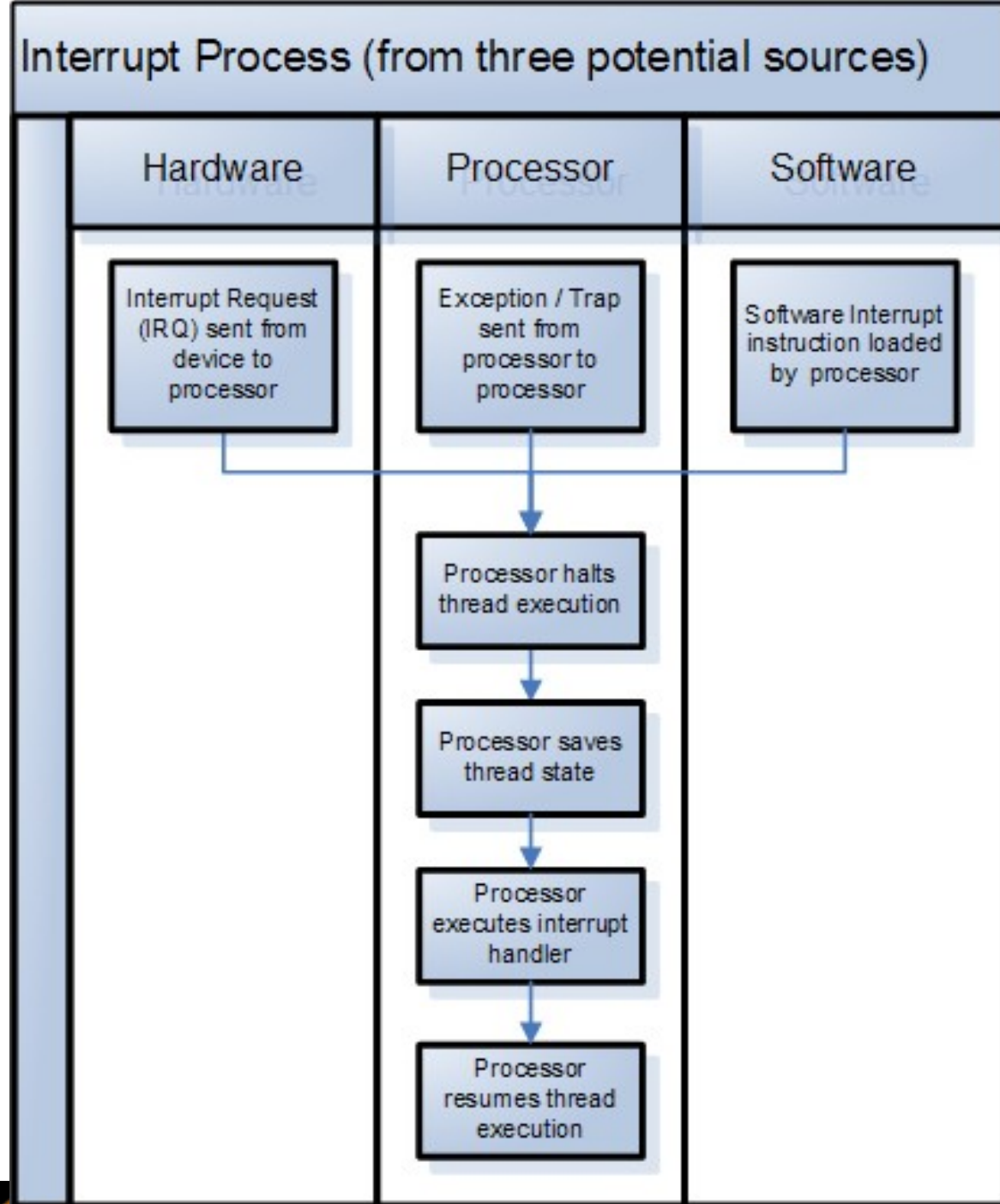
- How to manage concurrent operation of all these elements?
- An operating system is **interrupt** driven
- An interrupt is a signal sent to the operating system, for example:
  - Hardware interrupt by one of the devices
  - Software interrupt generated by user request
  - Trap or exception generated by an error
- The operating system has to listen to interrupts and react to them

## Operating System is Interrupt Driven





## Interrupts (2)

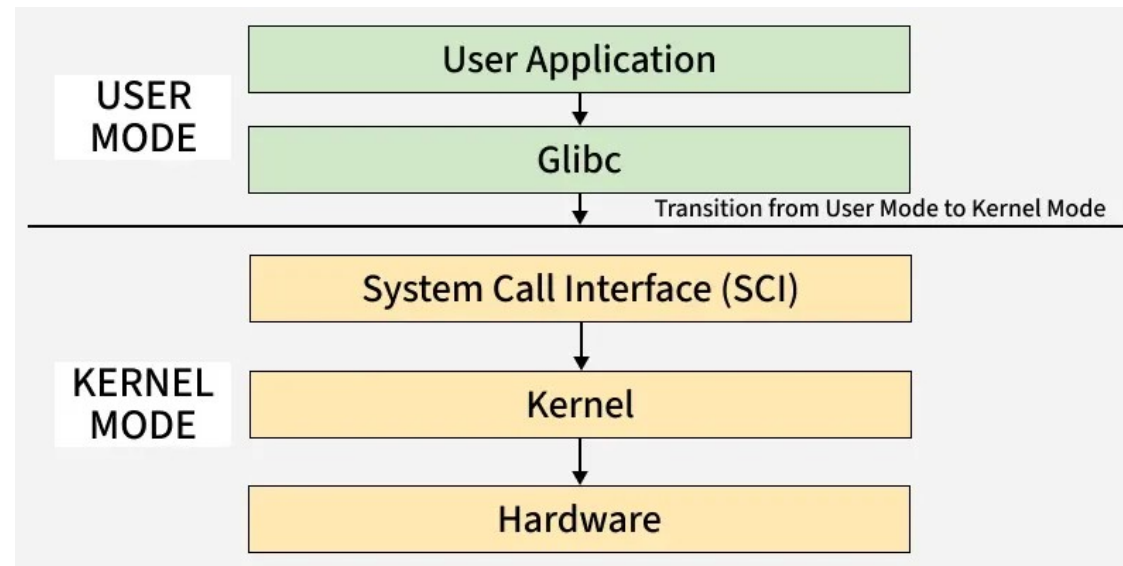






## Dual-Mode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - Hardware supports switching between modes
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions are *privileged*, i.e., only executable in kernel mode
    - *System call* changes mode to kernel, return from call resets it to user



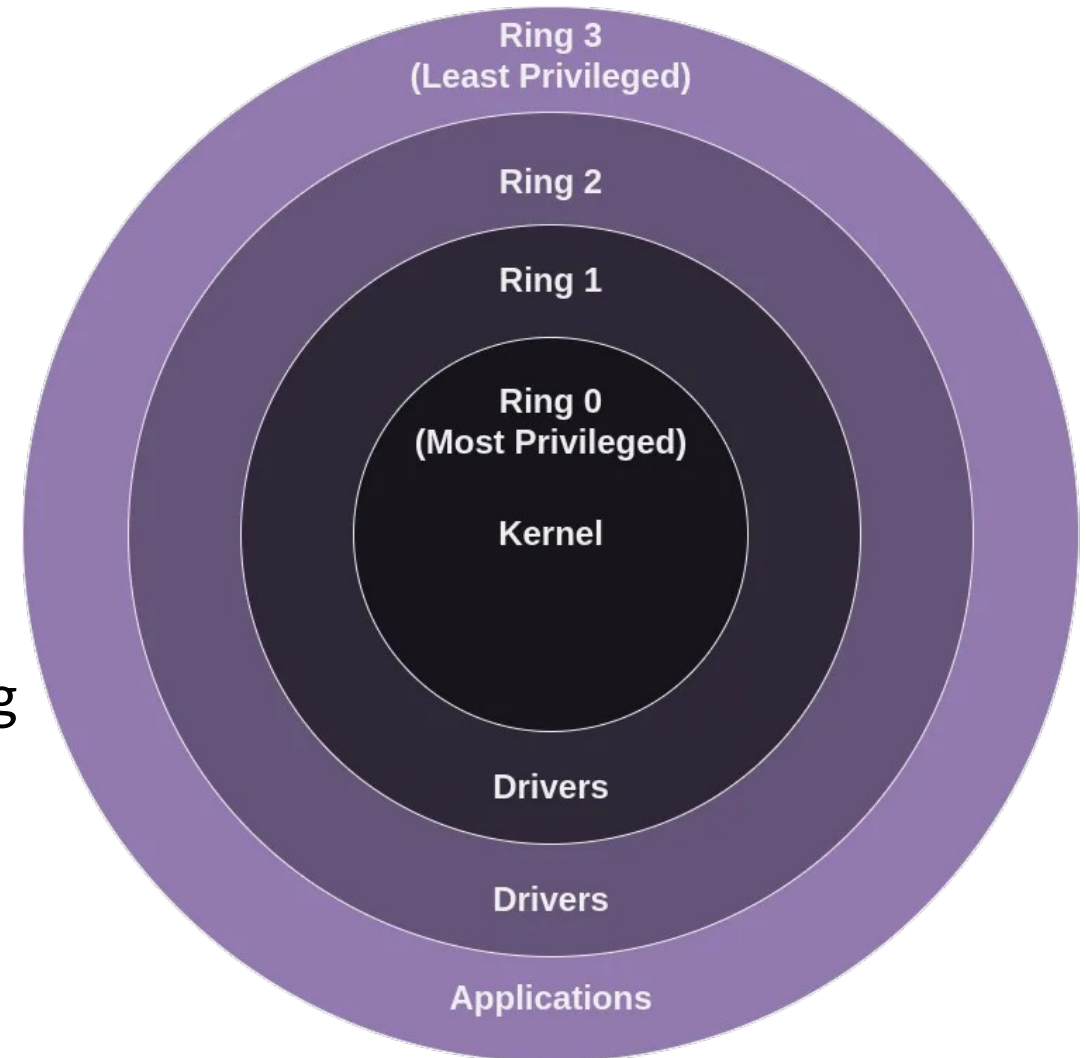


## Kernel mode

**Kernel mode** is a privileged mode of operation that allows the operating system (OS) to have **full control** over all hardware and system resources.

### Definition:

Kernel mode is the mode in which the OS runs when performing **critical tasks**—like interacting with hardware, managing memory, and handling system calls.





## Kernel mode

- **What It Can Do:**
  - Direct access to **all hardware components** (CPU, memory, disk, etc.)
  - Execute **privileged instructions** (e.g., accessing I/O ports, configuring interrupt tables)
  - Switch between **user processes**
  - Manage **security and resource allocation**

## Kernel mode

- **How It's Used:**
  - The OS **enters kernel mode** when:
    - A system call is made (e.g., opening a file)
    - An interrupt or exception occurs
    - It needs to schedule or manage processes
  - After completing the task, the OS **returns to user mode** to continue running the application safely.

## Kernel mode

- **Why It Matters:**
  - Prevents user applications from directly accessing or damaging critical system components.

Provides **process isolation**, **security**, and **stability**.



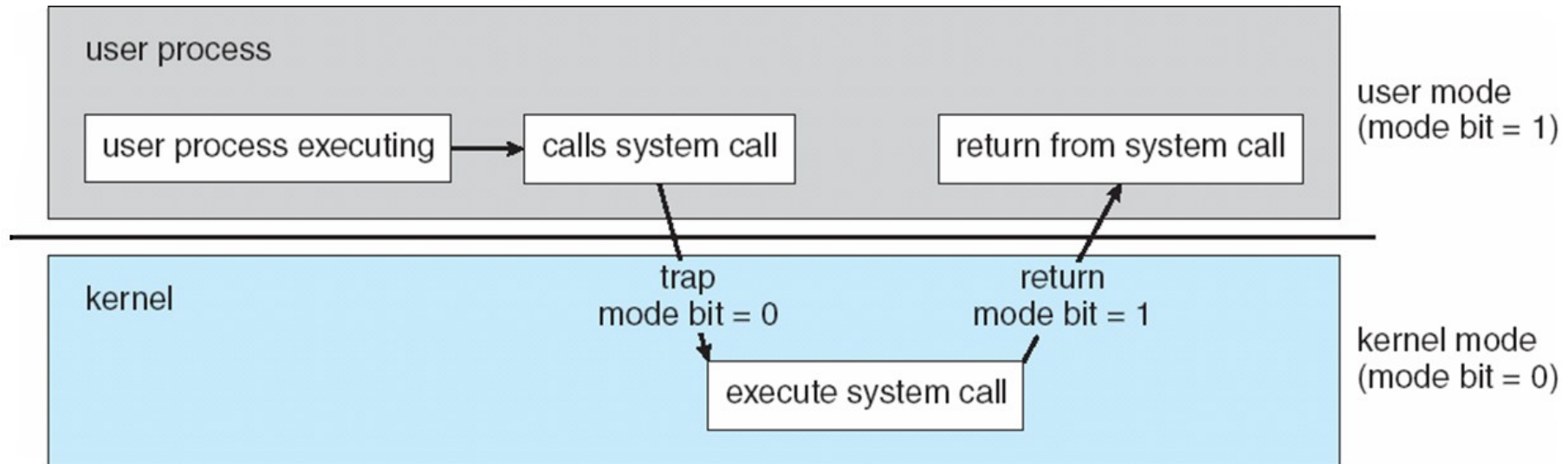


## User Mode and Kernel mode Summary Comparison

| Feature                 | User Mode          | Kernel Mode              |
|-------------------------|--------------------|--------------------------|
| Access to hardware      | ? No               | ? Yes                    |
| Privileged instructions | ? Not allowed      | ? Allowed                |
| Security risk           | Lower (restricted) | Higher (must be trusted) |
| Used by                 | Applications       | Operating System         |

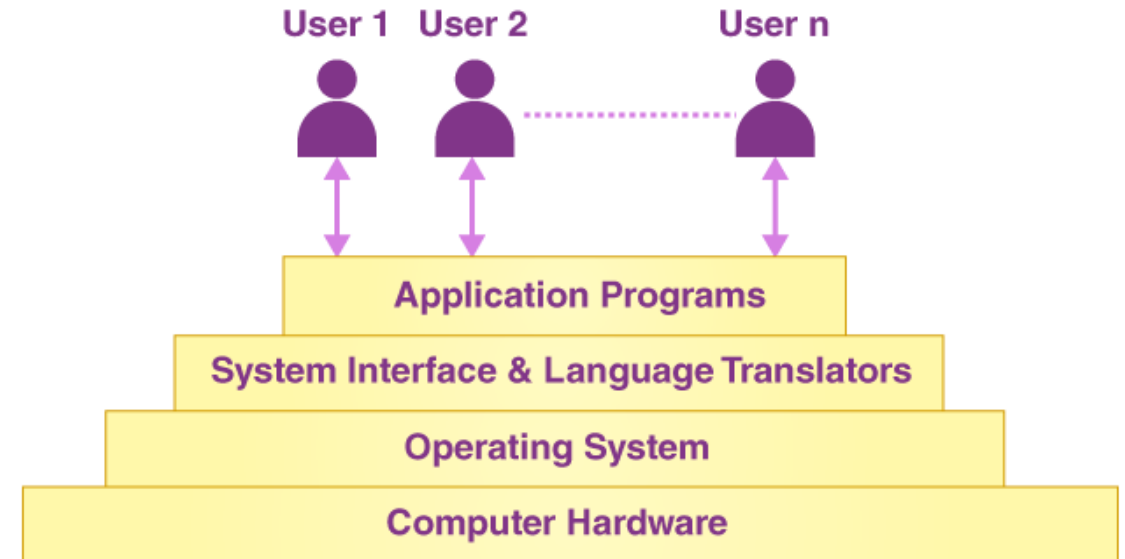


# Transition from User to Kernel Mode



# Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- Two sets of operating system services:
  1. Functions that are **helpful to the user**
  2. Functions to ensure the **efficient operation of the system itself**



Source: <https://byjus.com/gate/operating-system-services-notes/>

## User Services: User Interface

–Almost all operating systems have a user interface (UI)

Types of user interfaces:

Command Line Interface (CLI) or command interpreter allows direct command entry

Graphical user interface (GUI): User-friendly desktop metaphor interface

**Invented at Xerox Parc – guess when?**

Touchscreens: GUI with new challenges (no mouse, no keyboard)



## System Calls

Programming interface to the services provided by the Operating System

Mostly accessed by programs via a high-level **Application Programming Interface (API)**

Three common APIs are

- Win32 API for Windows
- POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)
- Java API for the Java virtual machine (JVM)
- Do you know of any other APIs?

# Example of Standard API

Go to terminal and  
type:  
`man read`

```
Terminal
Linux Programmer's Manual
NAME
  read - read from a file descriptor
SYNOPSIS
  #include <unistd.h>

  ssize_t read(int fd, void *buf, size_t count);
DESCRIPTION
  read() attempts to read up to count bytes from file descriptor fd into
  the buffer starting at buf.

  On files that support seeking, the read operation commences at the cur-
  rent file offset, and the file offset is incremented by the number of
  bytes read. If the current file offset is at or past the end of file,
  no bytes are read, and read() returns zero.

  If count is zero, read() may detect the errors described below. In the
  absence of any errors, or if read() does not check for errors, a read()
  with a count of 0 returns zero and has no other effects.

  If count is greater than SSIZE_MAX, the result is unspecified.

RETURN VALUE
  On success, the number of bytes read is returned (zero indicates end of
  file), and the file position is advanced by this number. It is not an
  error if this number is smaller than the number of bytes requested;
  this may happen for example because fewer bytes are actually available
  right now (maybe because we were close to end-of-file, or because we
  are reading from a pipe, or from a terminal), or because read() was
  interrupted by a signal. On error, -1 is returned, and errno is set
  appropriately. In this case, it is left unspecified whether the file
  position (if any) changes.

ERRORS
  EAGAIN The file descriptor fd refers to a file other than a socket and
  has been marked nonblocking (O_NONBLOCK), and the read would
  block.

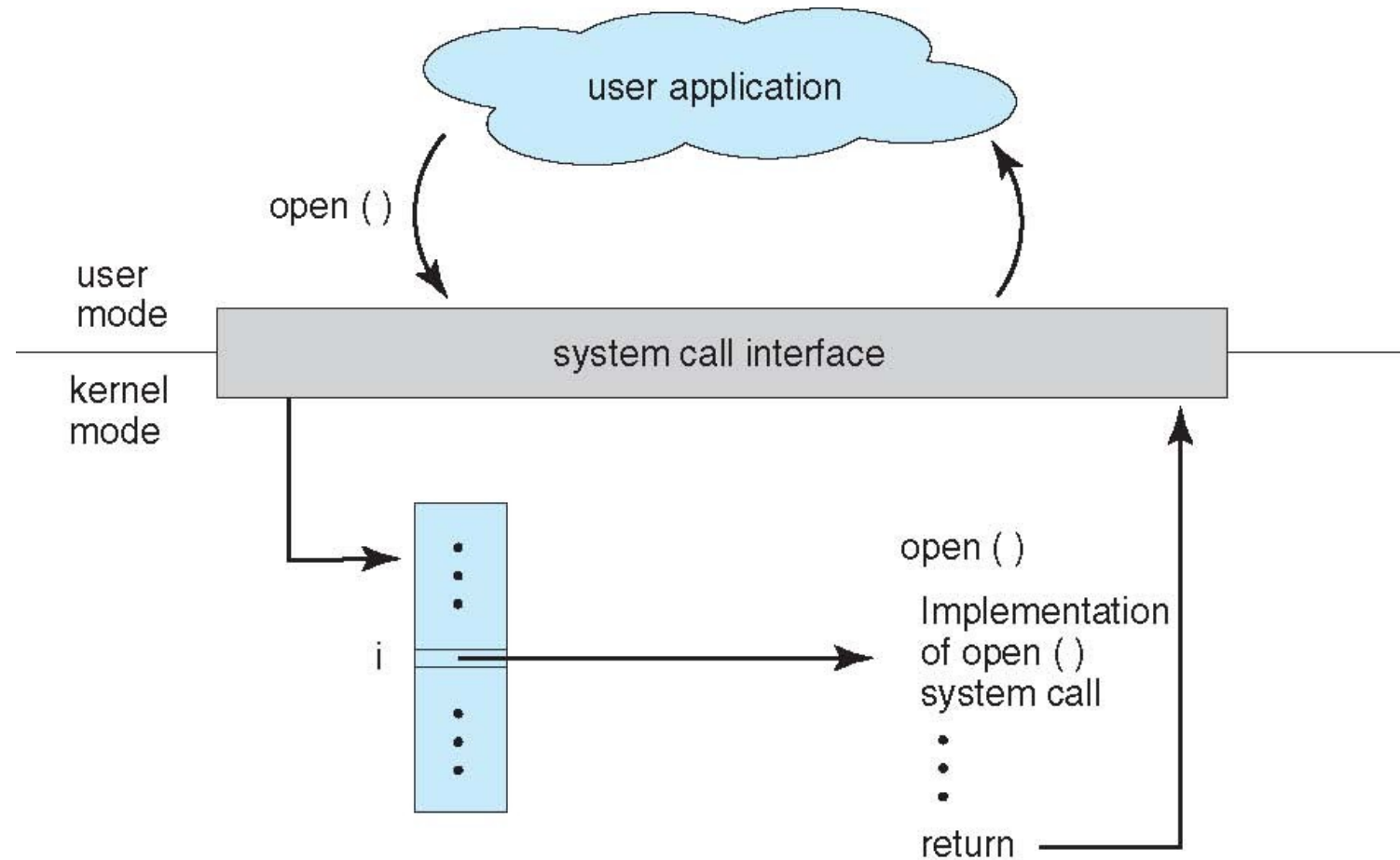
  EAGAIN or EWOULDBLOCK The file descriptor fd refers to a socket and has been marked
  nonblocking (O_NONBLOCK), and the read would block.
  POSIX.1-2001 allows either error to be returned for this case,
  and does not require these constants to have the same value, so

Manual page read(2) line 1/124 38% (press h for help or q to quit)
```





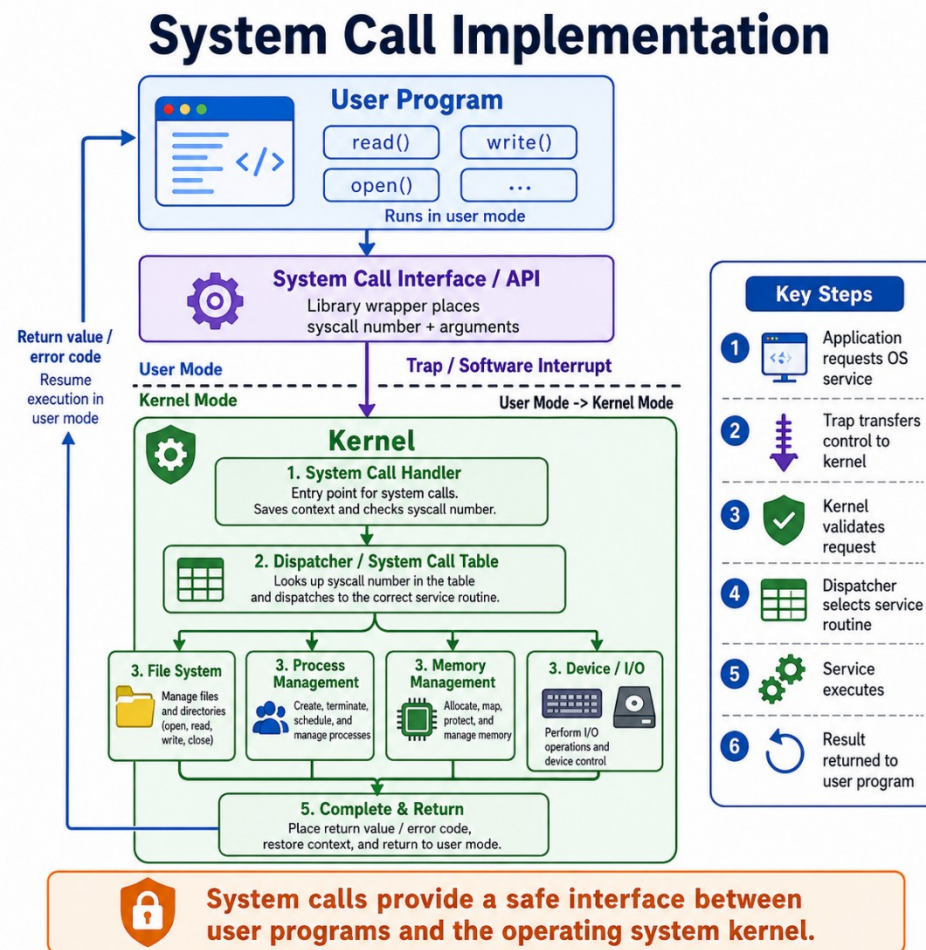
# API – System Call – OS Relationship





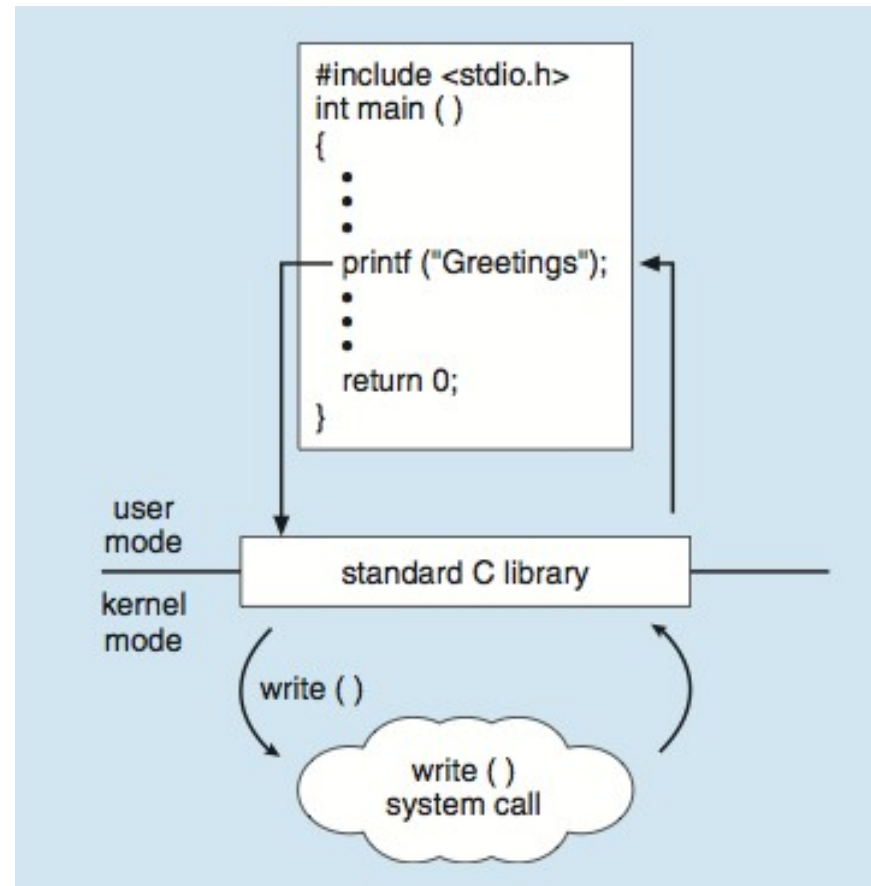
# System Call Implementation

- How does the operating system provide system calls?
- The system call **interface**
  - **invokes** the intended system call in OS kernel
  - **returns** status of the system call + return values
- The **caller** need know nothing about how the system call is implemented
  - Just needs to **obey API** and understand what OS will do as a result of the call
  - Most details of OS interface hidden from programmer by API



## Example: printf() in C

C program invoking printf() library call, which calls write() system call





## Types of System Calls

You are likely to find these types of system calls in **every** OS!

- Process control
- File management
- Device management
- Protection
- Information maintenance
- Communication

Notice the similarities to the Operating System services we discussed earlier?

# Examples of Windows + Unix System Calls

|                         | Windows                        | Unix     |
|-------------------------|--------------------------------|----------|
| Process Control         | CreateProcess()                | fork()   |
|                         | ExitProcess()                  | exit()   |
|                         | WaitForSingleObject()          | wait()   |
| File Manipulation       | CreateFile()                   | open()   |
|                         | ReadFile()                     | read()   |
|                         | WriteFile()                    | write()  |
|                         | CloseHandle()                  | close()  |
| Device Manipulation     | SetConsoleMode()               | ioctl()  |
|                         | ReadConsole()                  | read()   |
|                         | WriteConsole()                 | write()  |
| Information Maintenance | GetCurrentProcessID()          | getpid() |
|                         | SetTimer()                     | alarm()  |
|                         | Sleep()                        | sleep()  |
| Communication           | CreatePipe()                   | pipe()   |
|                         | CreateFileMapping()            | shmget() |
|                         | MapViewOfFile()                | mmap()   |
| Protection              | SetFileSecurity()              | chmod()  |
|                         | InitializeSecurityDescriptor() | umask()  |
|                         | SetSecurityDescriptorGroup()   | chown()  |



## System Programs

System programs use system calls to provide the functionality to the user

They can be divided into:

- File manipulation
- Status information
- Programming language support
- Program loading and execution
- Communications

Most users' view of the operating system is defined by system programs, not the actual system calls

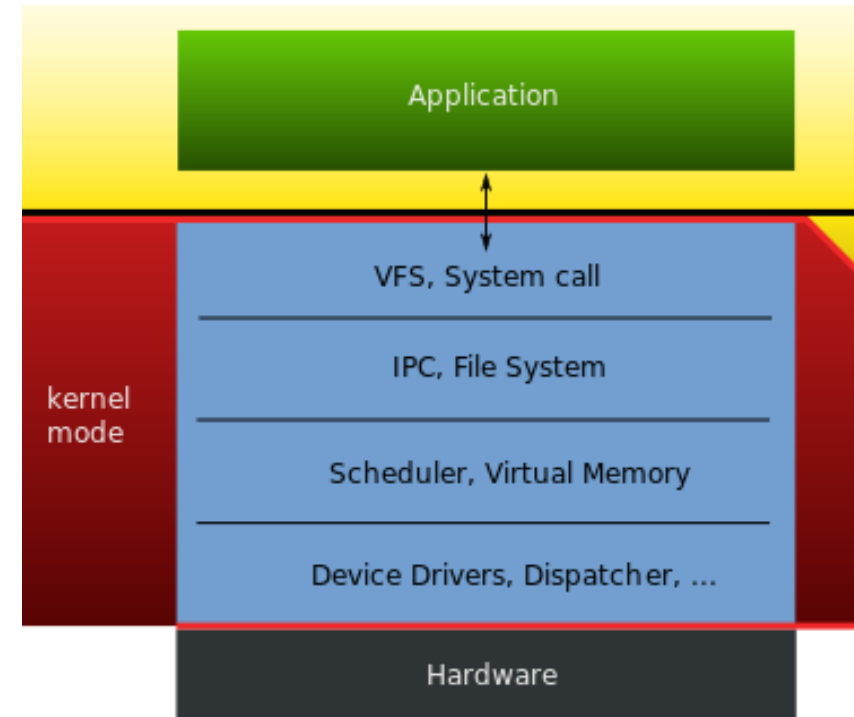


# OS Designs



## Monolithic Structure

- General-purpose OS is very large program
- Simplest way to structure is monolithic structure
- Example: MS-DOS
  - written to provide the most functionality in the least space
- Not divided into modules
- Interfaces and levels of functionality are not well separated



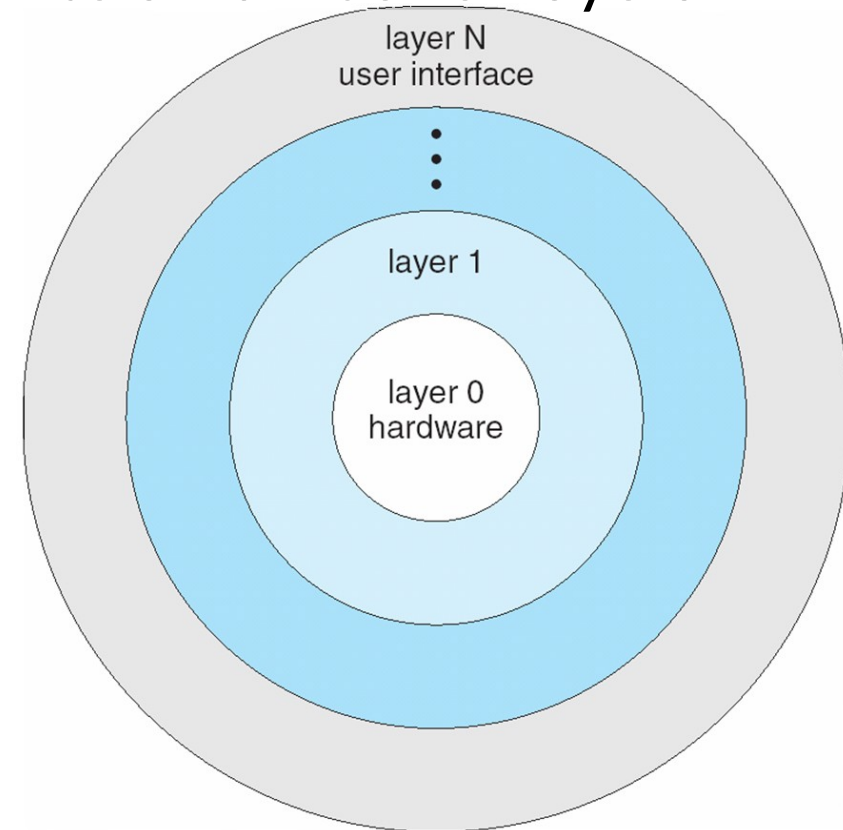
## Layered Structure

The operating system is divided into a number of layers (levels)

Each layer is built on top of lower layers.

The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.

Each layer uses functions (operations) and services of only lower-level layers

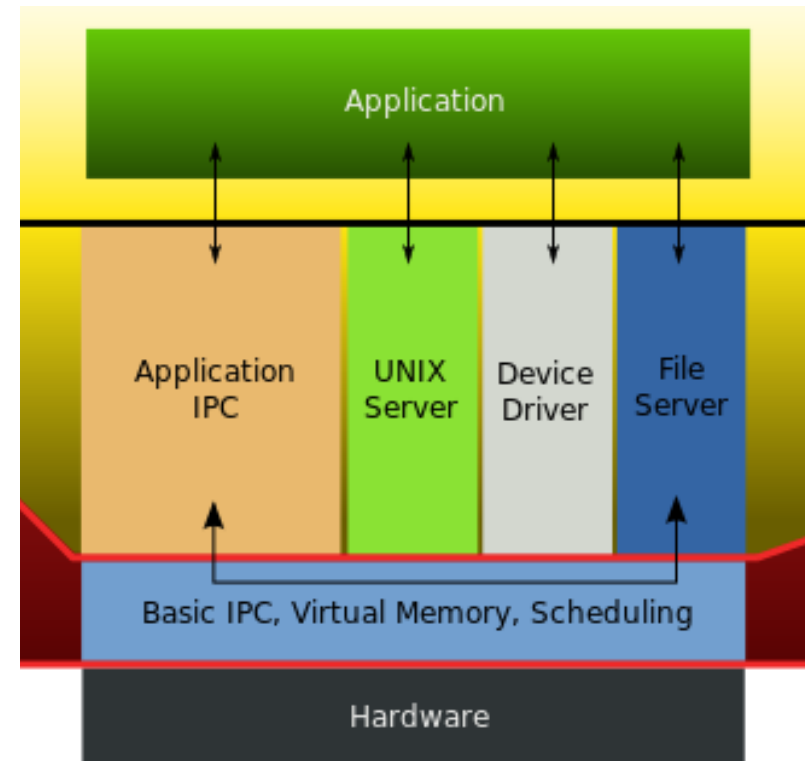


## Microkernel Structure

Moves as much as possible from the kernel into user space

Benefits: Easier to extend, more reliable (less code is running in kernel mode), more secure

But: performance overhead of user space to kernel space communication



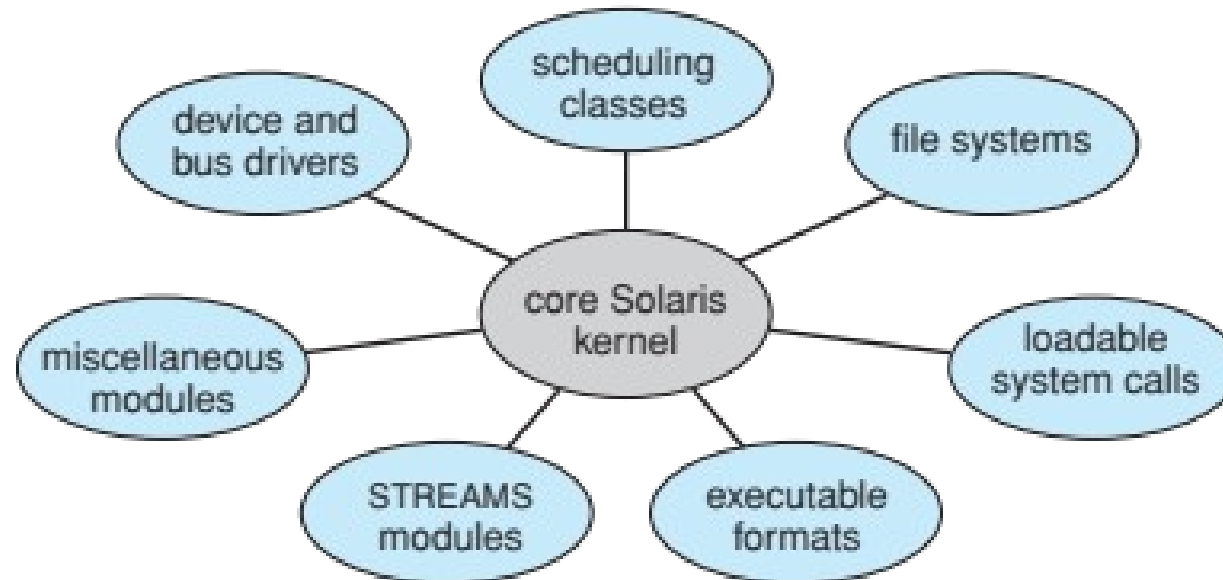


# Loadable Kernel Modules

Many modern operating systems implement **loadable** kernel modules

- Uses object-oriented approach
- Each core component is separate
- Each talks to the others over known interfaces
- Each is loadable as needed within the kernel

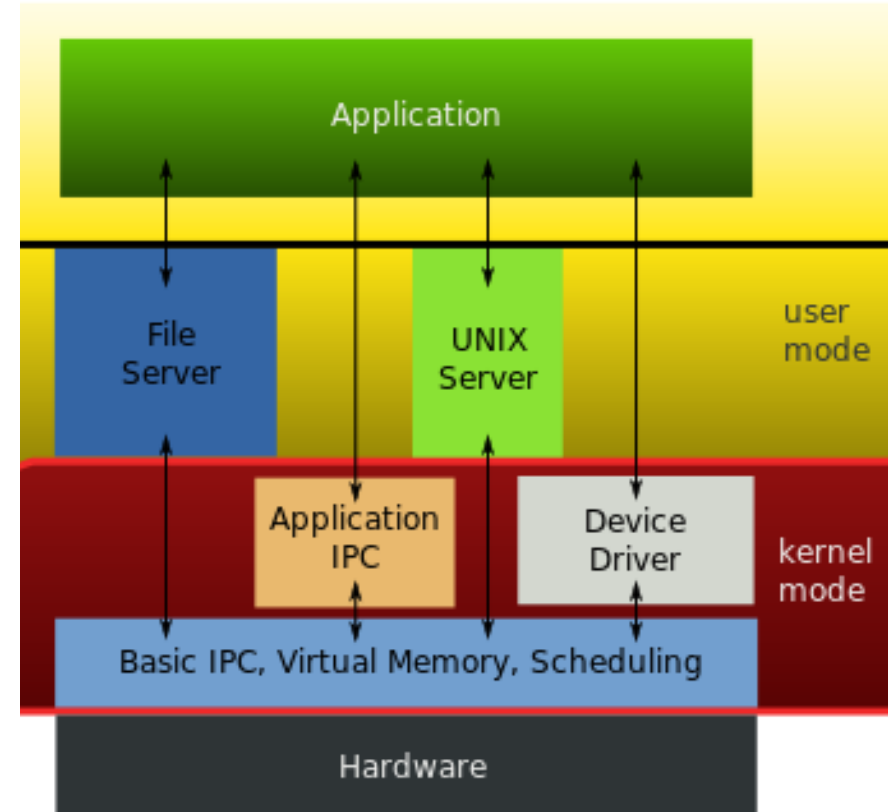
Similar to layers but more flexible



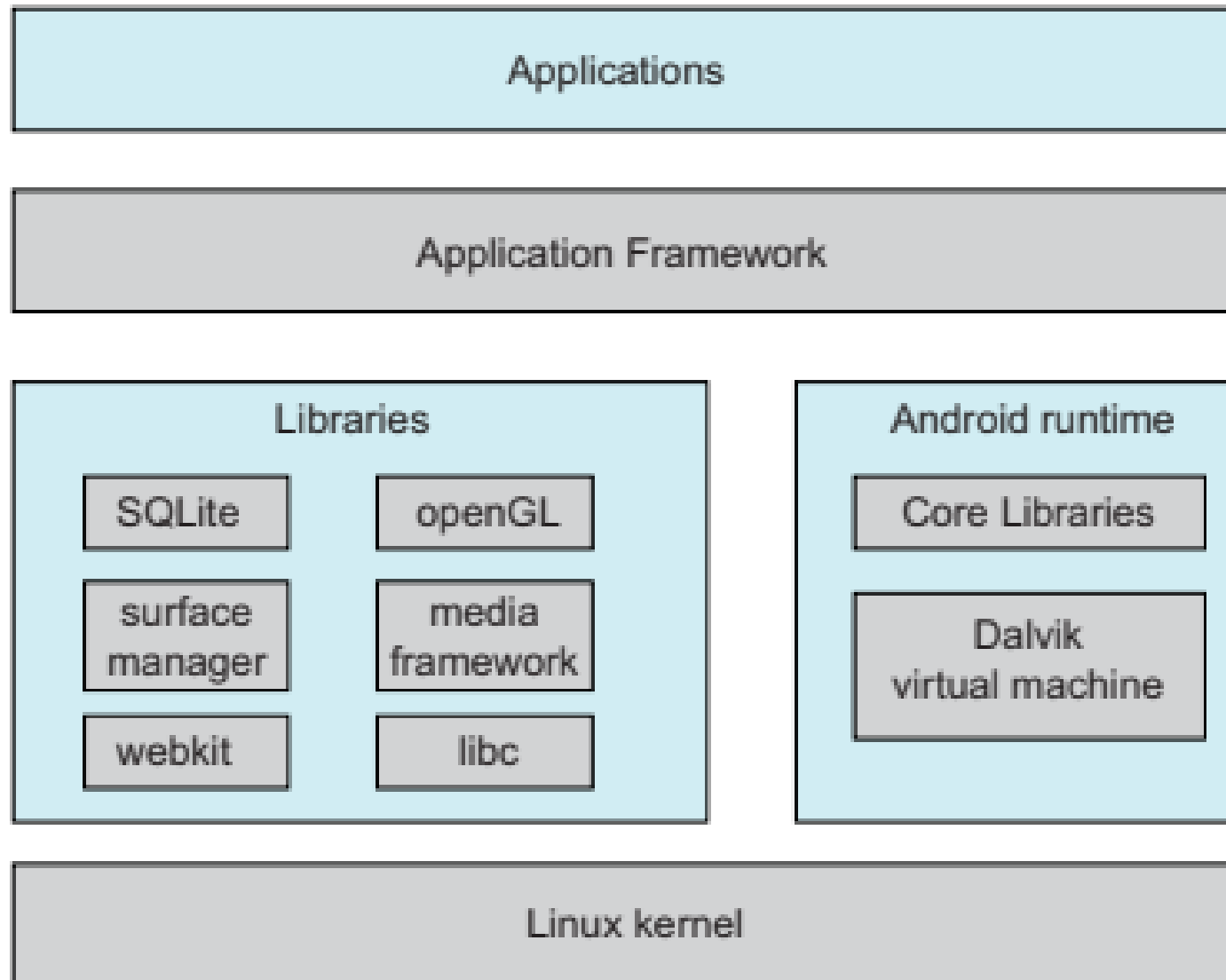


# Hybrid Systems

- Most modern operating systems are hybrid systems
  - Hybrid systems combine multiple approaches to address performance, security, usability needs
    - **Linux**: monolithic kernel plus modules
    - **Windows**: monolithic plus microkernel
    - **Android**: layered stack of software on top of Linux kernel









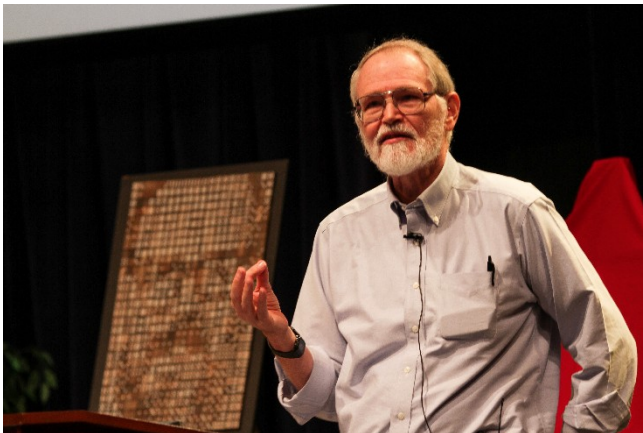
# Operating-System Debugging

OS generates **log files** containing error information

Failure of an application can generate **core dump** file capturing memory of the process

Operating system failure can generate **crash dump** file containing kernel memory

Kernighan's Law:



“Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.”



## System Boot in Three Steps

1. When system is powered on, execution starts at a **predefined memory location**
  - Firmware ROM (read-only memory) holds initial bootstrap loader
2. Operating system must be made available to hardware so hardware can start it
  - **Bootstrap loader** locates the kernel, loads it into memory, and starts it
  - Bootloader GNU GRUB allows selection of kernel from multiple disks, versions, kernel options
3. Kernel loads and system is then running

Files

# What are Files?

Contiguous logical address space  
Contents defined by file's creator  
What file types do you know?





# File Attributes

- Information about files
- Kept in the directory structure on disk
- Linux: use **ls** command to display file attributes

```
ls -@lGhisO
```

```
7984647    16 -rw-r--r--    1 lipika  staff  -   4.9K   6 Oct 11:44 archive-file.zip
7990238     0 drwxr-xr-x   46 lipika  staff  -   1.5K   8 Oct 14:28 architecture
7984973     0 drwxr-xr-x   62 lipika  staff  -   2.1K   5 Nov 20:23 networking
7984537     0 drwxr-xr-x   79 lipika  staff  -   2.6K  13 Nov 17:51 operating-systems
9525414   144 -rwxr-xr-x    1 lipika  staff  -    71K  26 Oct 18:57 schedule.ods
5409356  5320 -rw-r--r--    1 lipika  staff  -   2.6M   9 Aug 08:50 some-program.exe
```

unique  
serial  
number  
(inode)

number of  
blocks on  
disk

access control  
list: who can read,  
write, execute

number  
of links

owner+group

file size

modification  
date+time

file name



## File and Directory Operations

File is an abstract data type

Regardless of the file contents, we can use generic operations on files, for example:

- Create, delete
- Write – at write pointer location
- Read – at read pointer location
- Seek – Reposition within file

Similarly, we can use generic operations on directories:

Search for a file

Create a file, delete a file

List a directory

Rename a file

Traverse the file system





# Directory Organization

Why directories?

Directories are organized logically to obtain

- Efficiency – locating a file quickly

- Naming** – convenient to users

  - Two users can have same name for different files

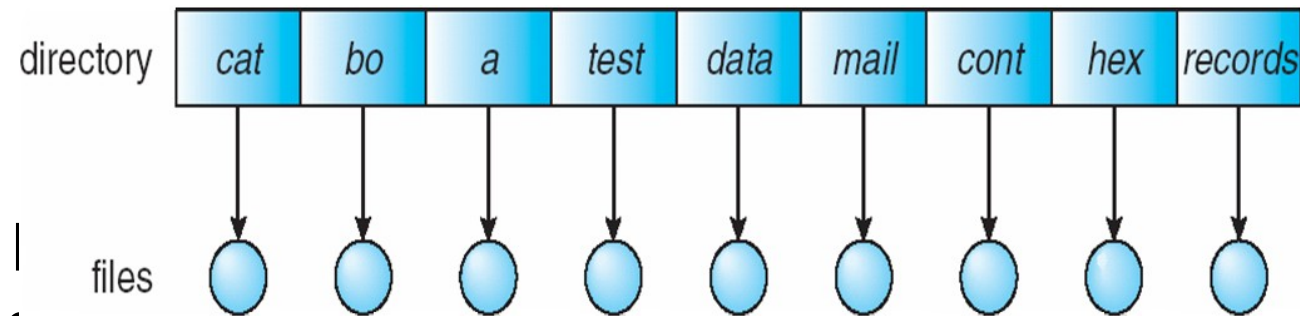
  - The same file can have several different names

- Grouping** – logical grouping of files by properties, (e.g., all Java programs, all games, all files related to a project, ...)



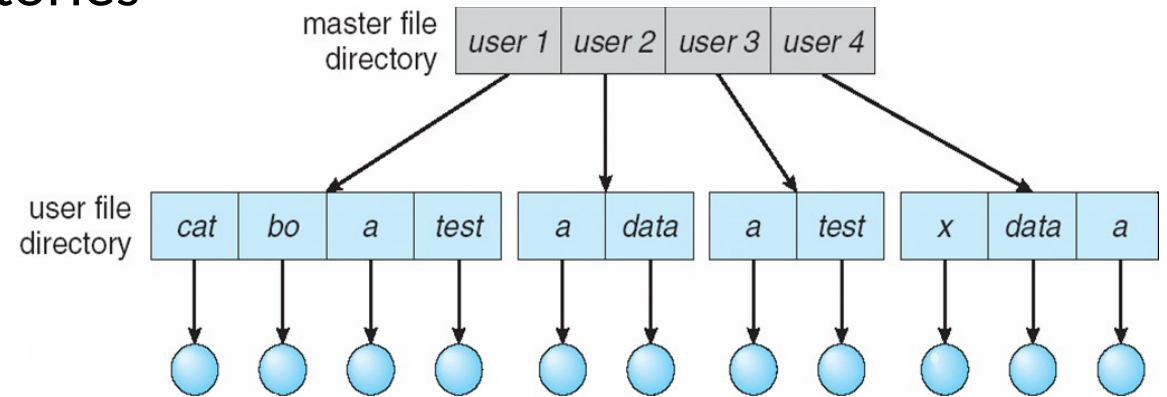
## Single-Level Directory

- One level of directories, no subdirectories
- Used historically and on some embedded systems



- Naming |
- Grouping problem

- Small improvement: two levels of directories

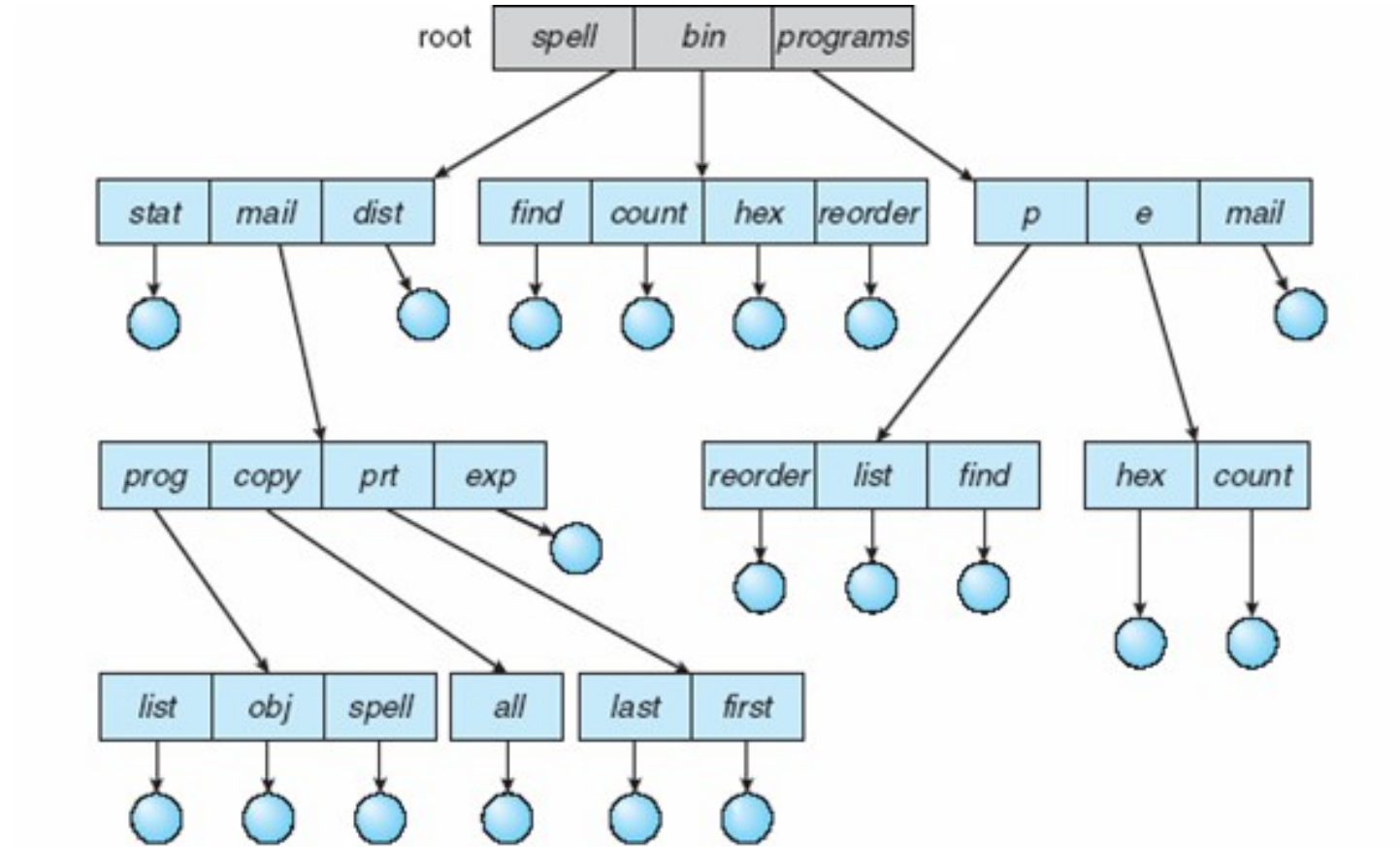


- Can have the same file name for different user
- Need path names to identify files, e.g. `/user1/cat`
- Efficient searching
- No grouping capability



# Tree-Structured Directories

Generalization: unlimited levels of directories



## Tree-Structured Directories (Cont.)

Efficient searching

Grouping Capability

Current directory (working directory)

```
cd /spell/mail/prog
```

```
pwd
```

Pathnames

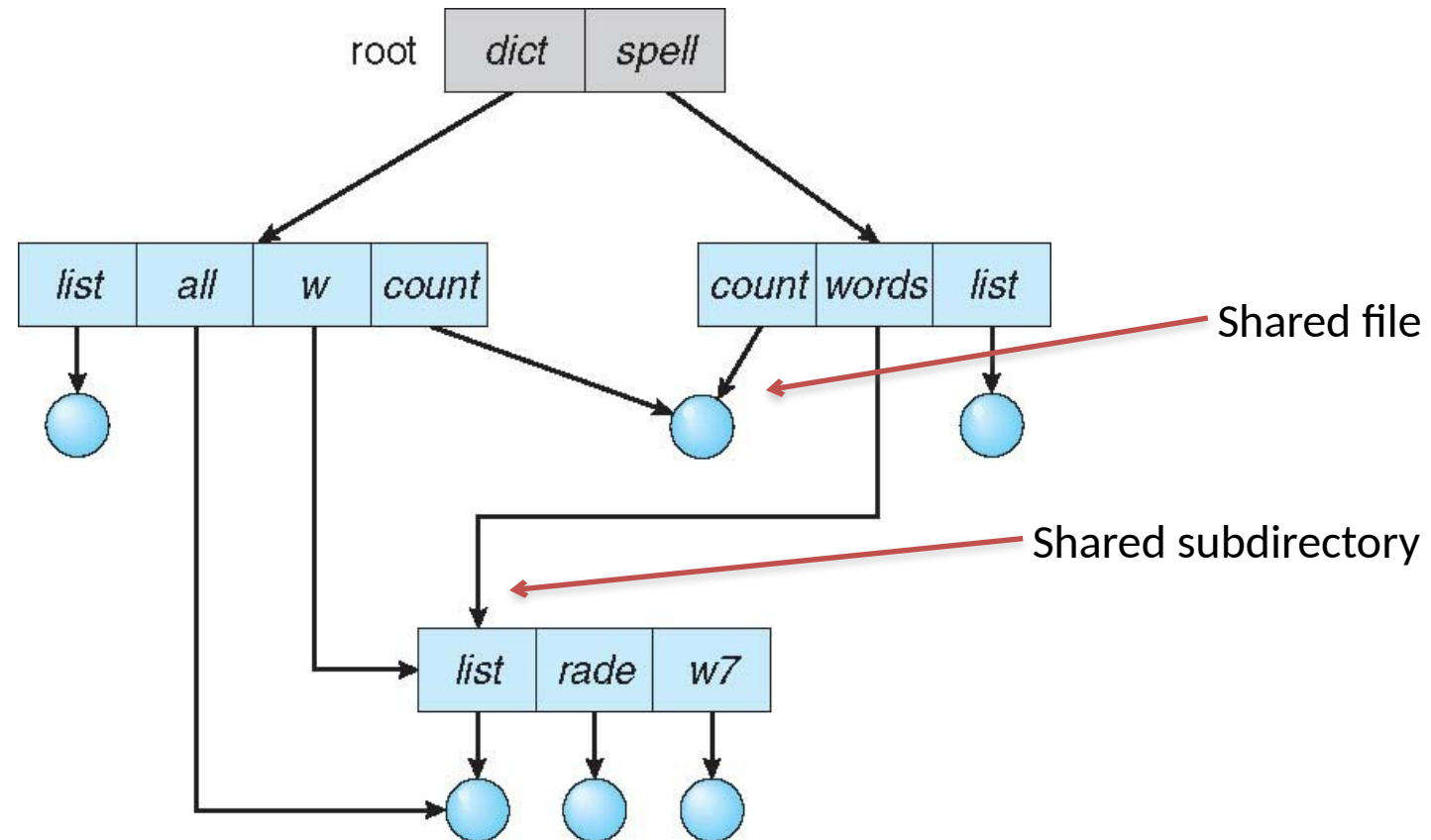
Absolute: includes all directories starting from root

Relative: starts from current working directory



# Acyclic-Graph Directories

In addition to tree structure: allow shared subdirectories and files





## Acyclic-Graph Directories (Cont.)

Need a new directory entry type: **link**

- another name (pointer) to an existing file
- Resolve the link – follow pointer to locate the file

Can lead to two different names for the same file/directory

Can lead to dangling pointers when file is deleted

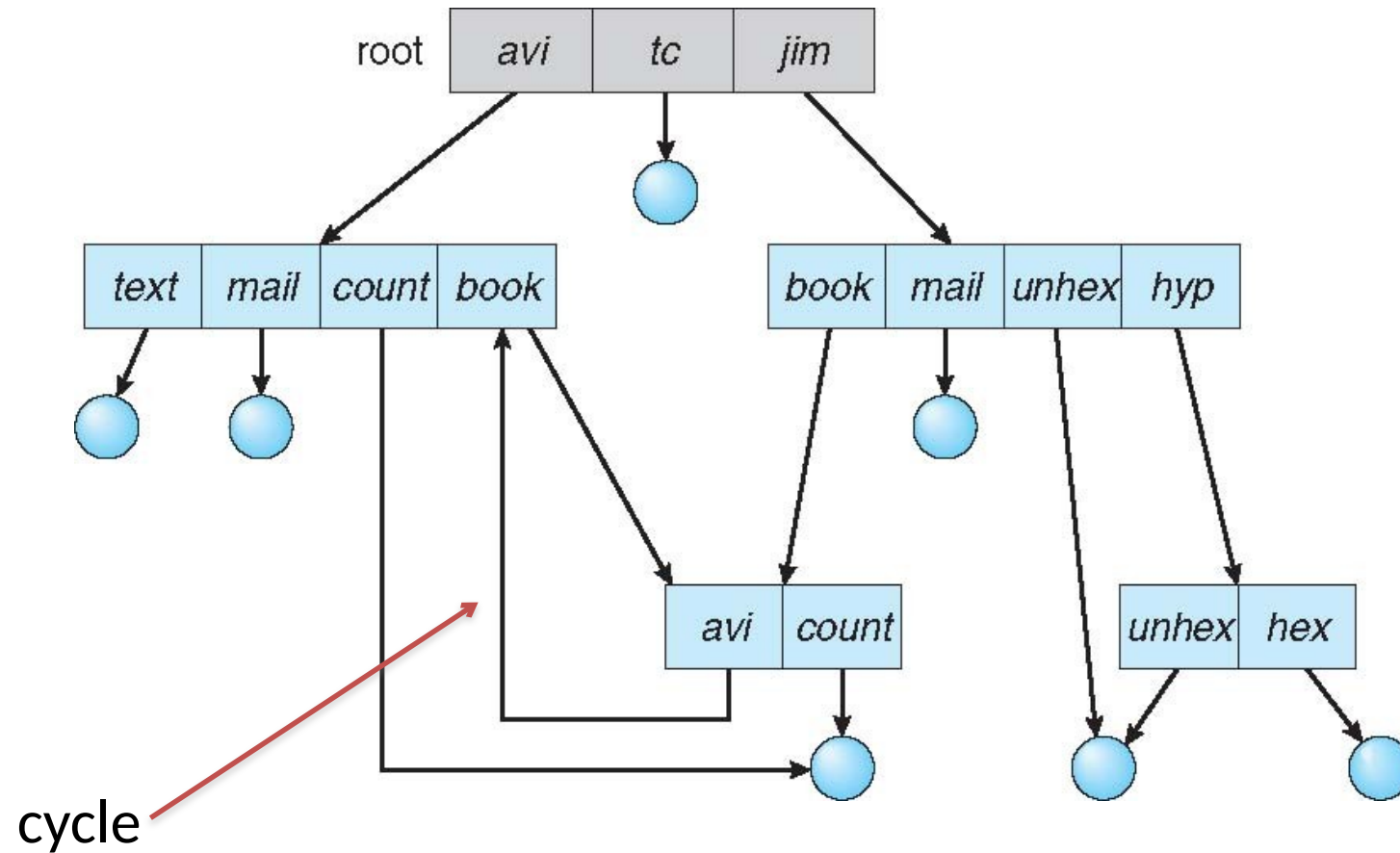
Solutions:

Backpointers, so we can delete all pointers

Entry-hold-count solution



# General Graph Directory





## General Graph Directory (Cont.)

- Cycles can be a problem: consider searching for a file in all directories
- How do we guarantee no cycles?
  - Allow only links to file not directories
  - Garbage collection
  - Every time a new link is added use a cycle detection algorithm to determine whether it is OK



# File Systems

Which file systems do you know?

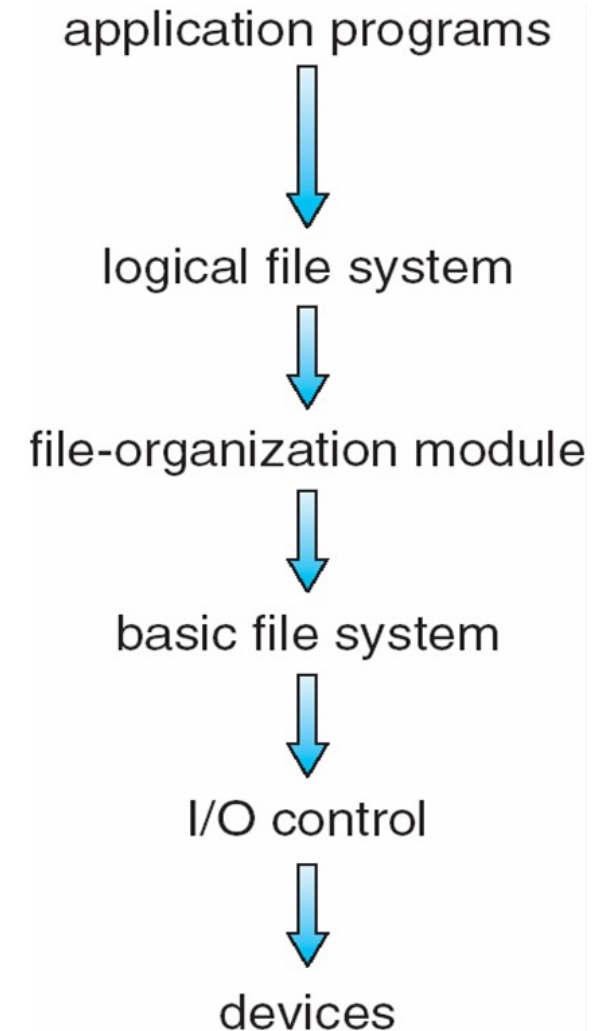
- Many different file systems exist, even within an operating system
  - CD-ROM is ISO 9660
  - **Windows** has FAT, FAT32, NTFS as well as floppy, CD, DVD, Blu-ray
  - **Linux** has extended file system ext2, ext3, ext4, supports many other file systems
  - **Distributed** (network) file systems: SMB, CIFS
  - **Clustered** file systems: Lustre
  - New ones still arriving, for example ZFS, GoogleFS, Oracle ASM, FUSE

How should we structure the operating system to work with file systems?

**Layering** useful for reducing complexity and redundancy

But: adds overhead and can decrease performance

Major challenge in designing operating systems: How many layers?





## File System Layer: Application

**Application programs** are the layer of the file system interface

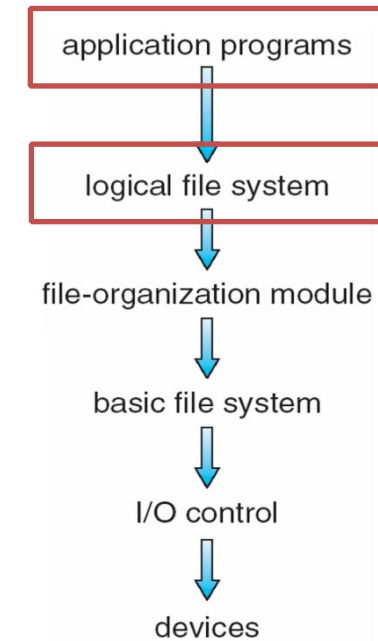
Applications request read/write operations using file names

**Logical file system** manages **metadata** information in file control blocks (called **inodes** in UNIX)

Translates file name into file number, file handle, location

Directory management

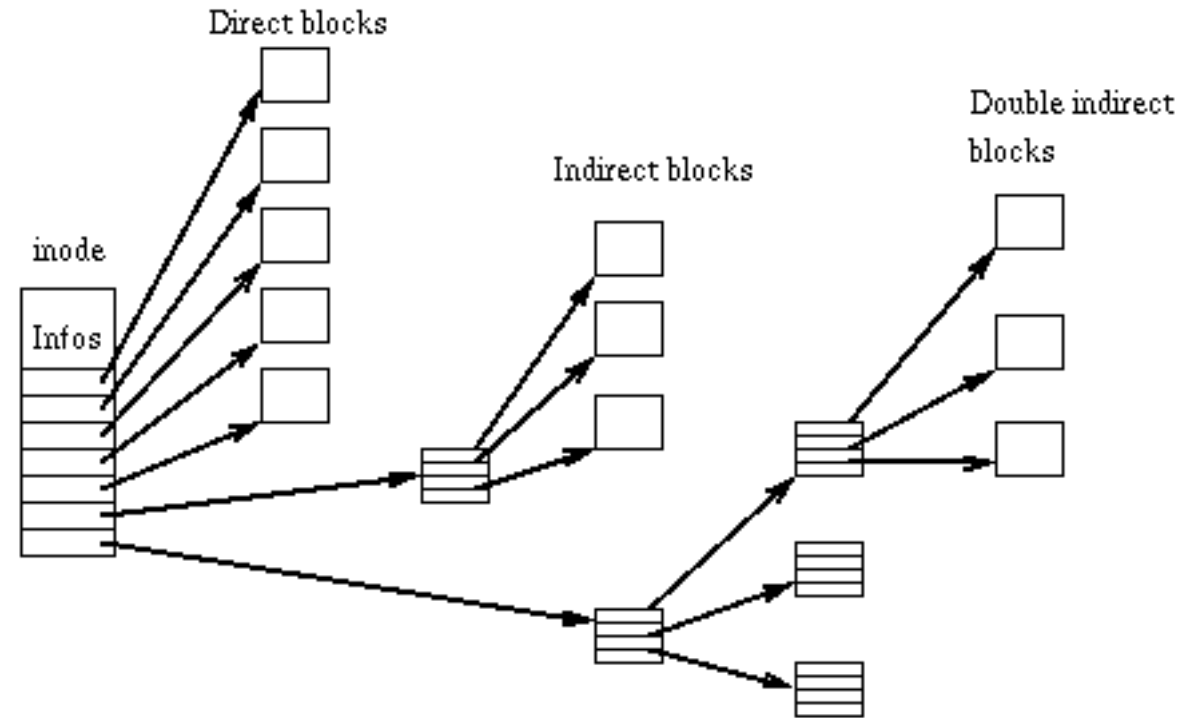
Protection





# inodes

File size  
Device id  
User, group id  
Access control list  
Timestamps  
    Last modified  
    Last accessed  
Link count  
Pointers to data blocks



- Use **stat** system call/system program to retrieve information from inode
- Indirect blocks support large files without large inodes

# File System Layer: File Organization

**File organization** module understands files (by file number), logical address, and physical blocks

Translates logical block # to physical block #

Manages **free space, disk allocation**

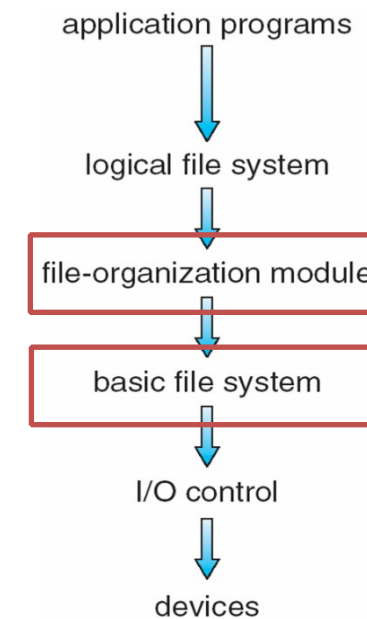
**Basic file system** works with device driver

Given command “retrieve (physical) block 123” translates to device driver

Also manages memory buffers and caches (allocation, freeing, replacement)

**Buffers** hold data in transit

**Caches** hold frequently used data

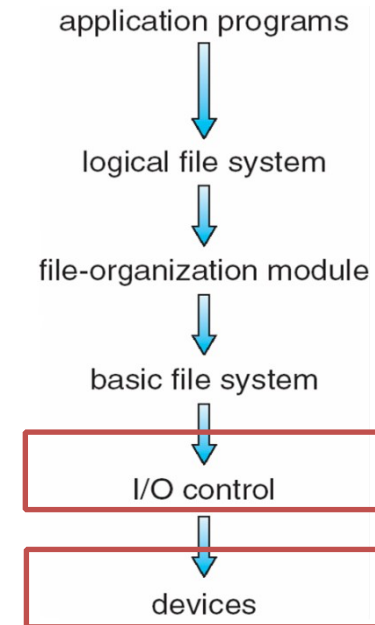




## File System Layer: I/O Control

**Device drivers** manage I/O devices at the I/O control layer

Given commands like “read drive 1, cylinder 72, track 2, sector 10, into memory location 1060” outputs low-level hardware specific commands to hardware controller





*Thank You!*